Lecture 5.2: AWS Database

Maharishi International University

Department of Computer Science

M.S. Thao Huy Vu

Maharishi International University - Fairfield, Iowa

All rights reserved. No part of this slide presentation may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying or recording, or by any information storage and retrieval system, without permission in writing from Maharishi International University (MIU).

Agenda

- **Introduction to NoSQL Services in AWS**
- **Amazon DynamoDB**
- **Amazon DocumentDB**
- **Amazon ElastiCache**
- **Amazon Neptune**

Introduction to NoSQL Services in AWS

- NoSQL vs. SQL
  - Collection – Table
  - Document – Row
  - Key – Column
  - Value – Column Value
- Scalability
- Flexibility
- Semi-structured or unstructured data

Introduction to NoSQL Services in AWS

- Amazon DynamoDB
- Amazon DocumentDB
- Amazon Keyspaces (for Apache Cassandra)

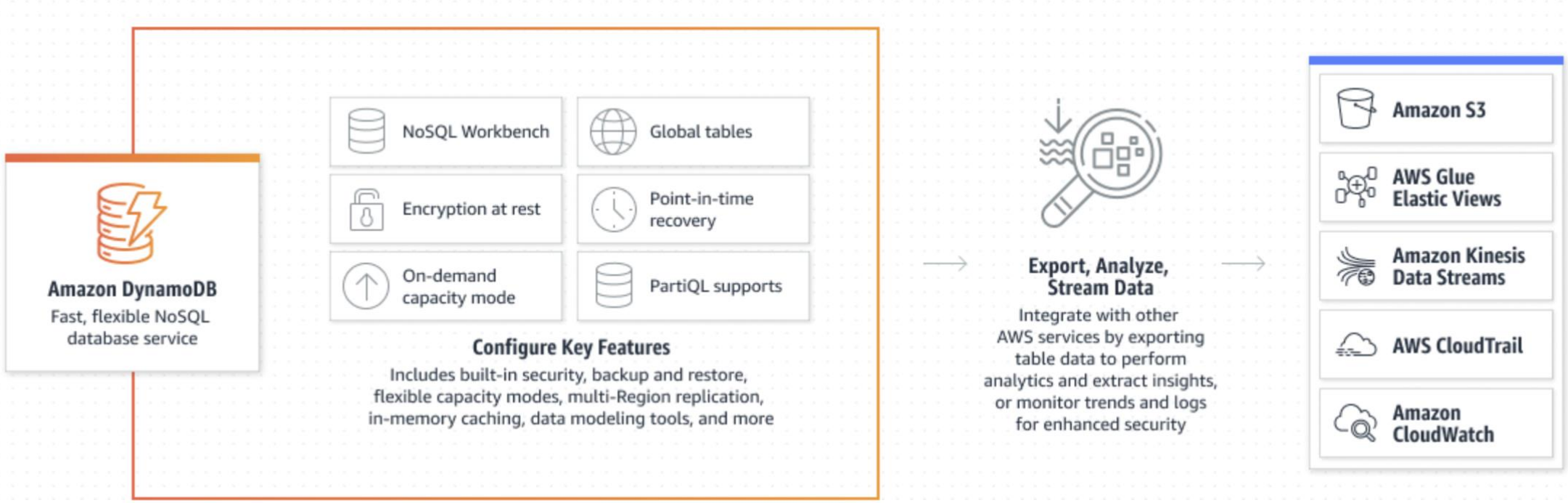
Introduction to NoSQL Services in AWS

- Factors to choose NoSQL database
  - Model: Key-value, document, wide-column.
  - Access patterns.
  - Performance requirements.
  - Specific application needs.
- Use cases for NoSQL database:
  - Web-scale Applications: DynamoDB.
  - Content Management Systems: DocumentDB.
  - High-write throughput systems: Keyspaces.

Types of NoSQL Databases

- **Key-value**: Key-value databases are highly partitionable and allow horizontal scaling at scales that other types of databases cannot achieve. Amazon DynamoDB is designed to provide consistent single-digit millisecond latency for any scale of workloads.
- **Document**: In application code, data is represented often as an object or JSON-like document because it is an efficient and intuitive data model for developers. Amazon DocumentDB (with MongoDB compatibility).
- **Graph**: A graph database's purpose is to make it easy to build and run applications that work with highly connected datasets. Typical use cases for a graph database include social networking, recommendation engines, fraud detection, and knowledge graphs. Amazon Neptune is a fully-managed graph database service.
- **In-memory**: Gaming and ad-tech applications have use cases such as leaderboards, session stores, and real-time analytics that require microsecond response times. Amazon ElastiCache offers Memcached and Redis, to provide low-latency, high-throughput.
- **Search**: Many applications output logs to help developers troubleshoot issues. Amazon Elasticsearch Service (Amazon ES) is purpose built for providing near-real-time visualizations and analytics of machine-generated data by indexing, aggregating, and searching semistructured logs and metrics.

Amazon DynamoDB



Amazon DynamoDB

- A fully managed NoSQL database service designed for document and key-value store models
- Seamless scalability
- High availability
- Predictable performance

Amazon DynamoDB

- Auto scalability
- Data model
- Indexing
- Read and Write Consistency Models

Amazon DynamoDB

- Provisioned and On-Demand Capacity Modes
- DynamoDB streams
- Backup and Restore
- Security

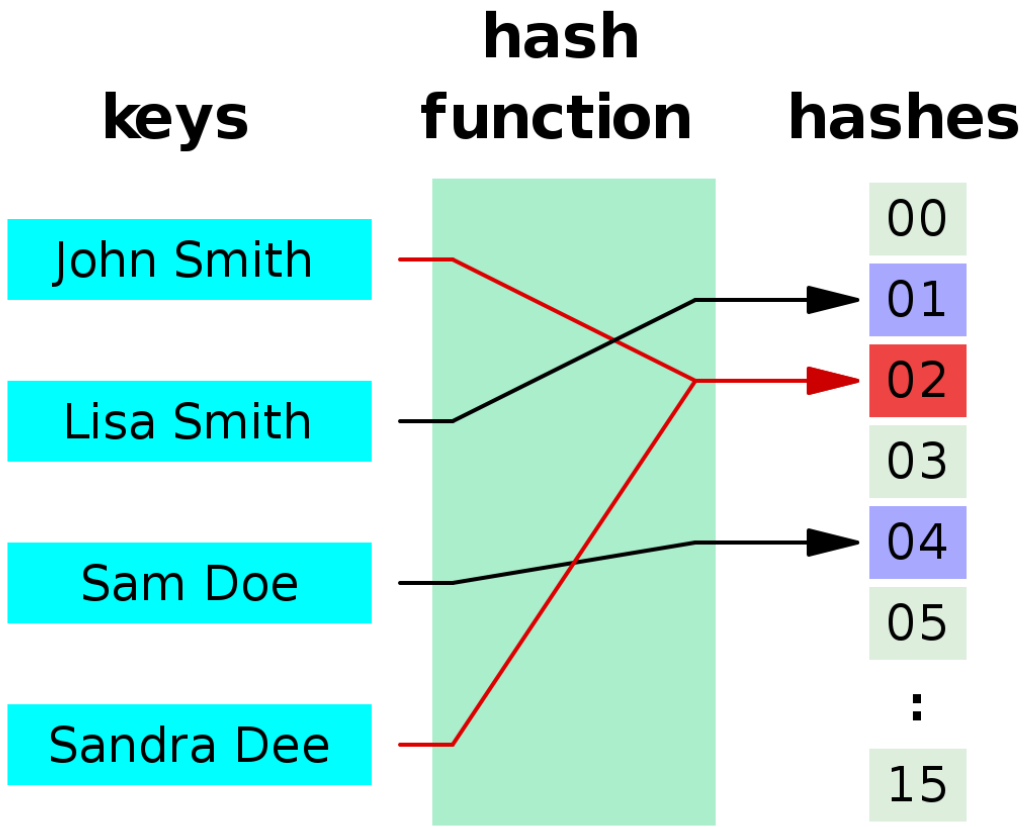
DynamoDB

- Flexible Table
- Partition Key: To determine the physical storage location of data. All items with the same partition key value are stored together which optimizes data retrieval.
- Sort Key: To determine the order of items within a partition. Items with the same partition key value are sorted by their sort key values.
- Primary Key: A combination of Partition Key and Sort Key

Partition key

It is a simple primary key.

DynamoDB uses the partition key's value as input to an internal **hash function**. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored.



The problem and power of sort (range) key

- In modern applications, performance at scale is one of the most important things.
- **The problem:** Relational databases are slow because you have to join tables to get data in most cases.
- **Solution:** Keep two tables in one. That is supported by modern databases such as DynamoDB, Casandra, MongoDB.
- In DynamoDB, the Sort or Range key is used to keep two tables in one.

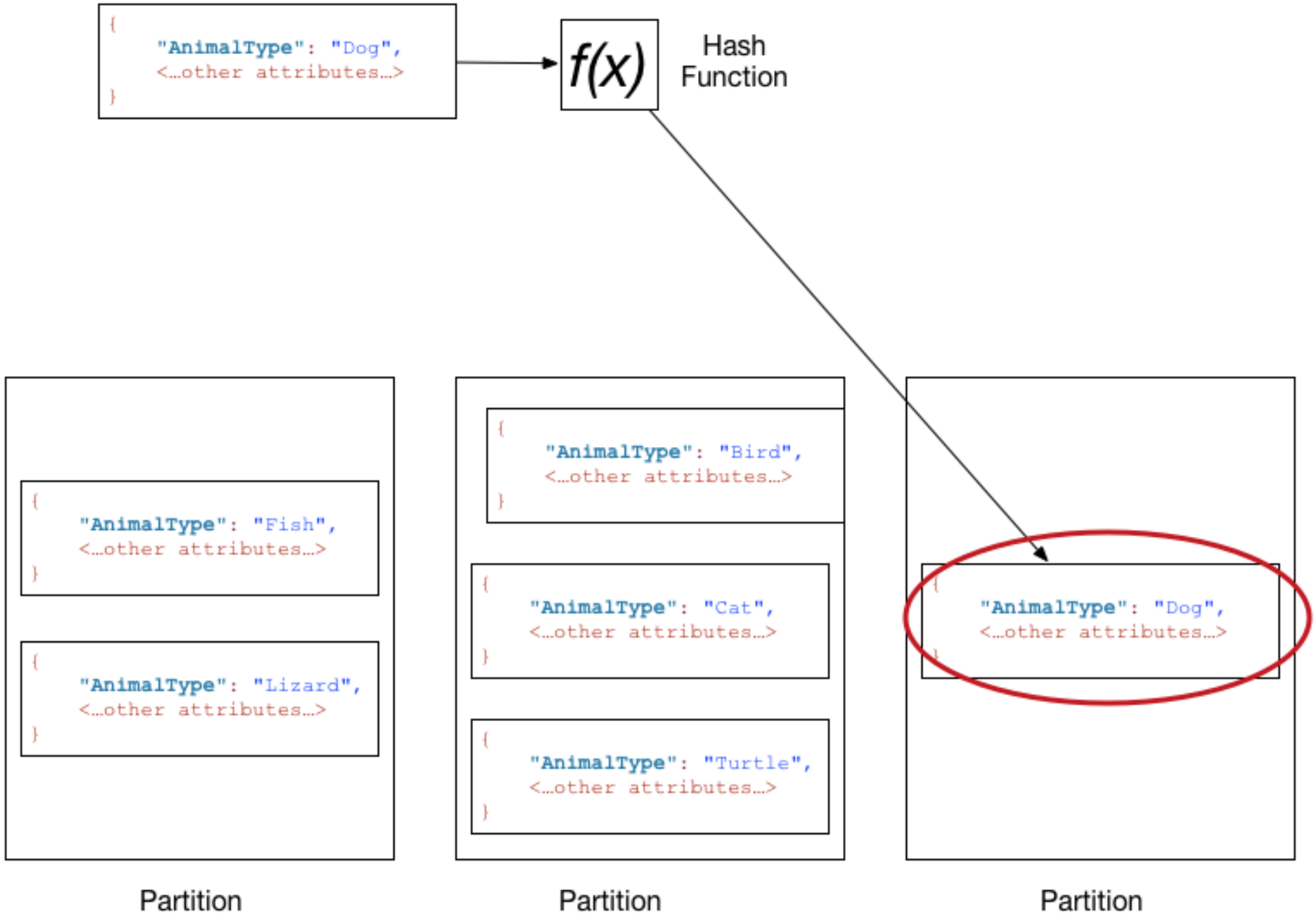
Sort (Range) key

- The sort key is used to represent the one-to-many relationship. For example, course and teacher. Course code as a partition key and teacher name as a sort key.
- You can use sort key to take advantage of sorting provided by DynamoDB.
- Partition key and sort key are a composite key that identifies a single item.

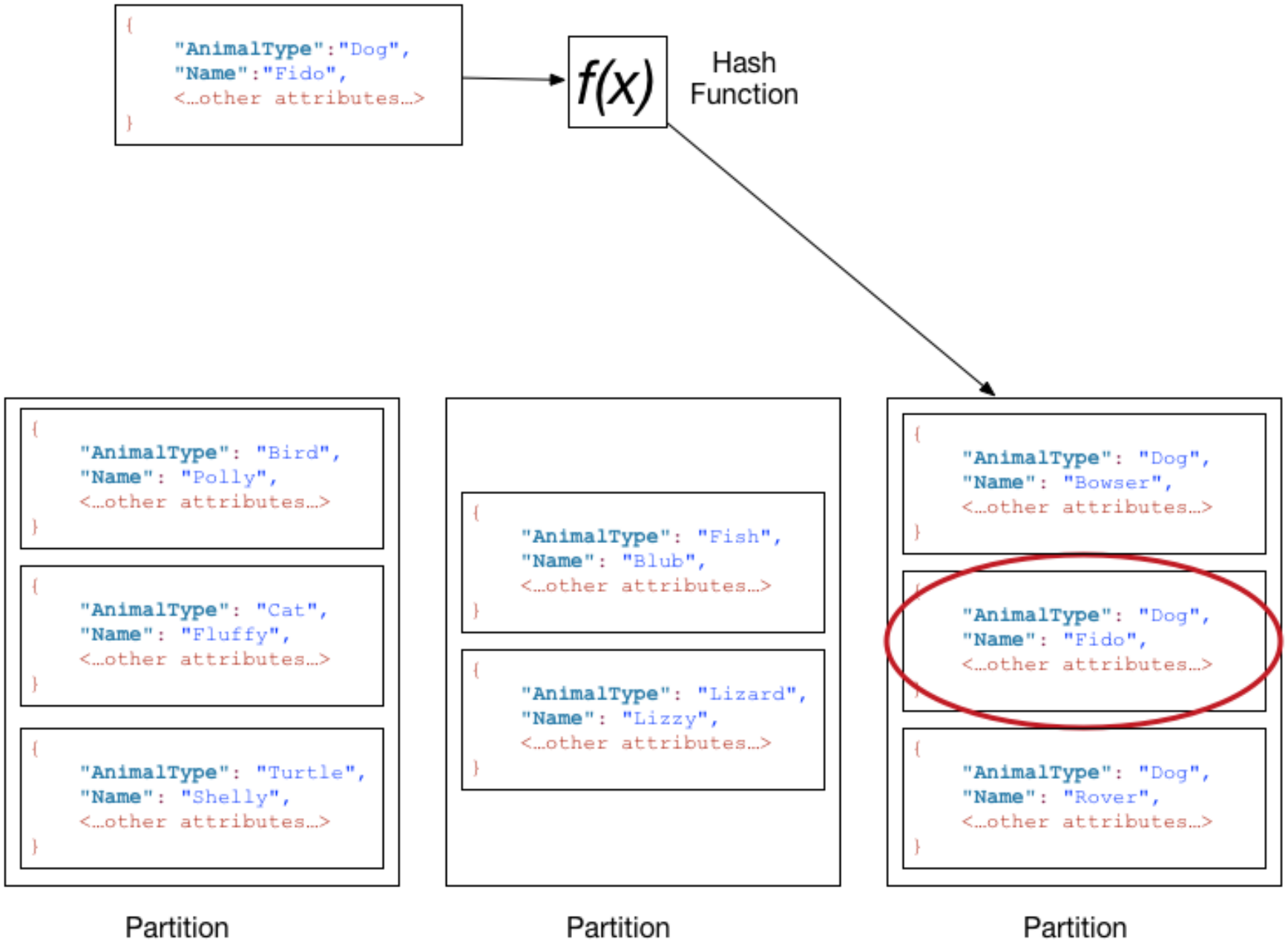
Composite primary key

A composite primary key is **composed of two attributes**. If you have a sort key in DynamoDB, that means you have a composite key. When get one item, you must provide both values, partition and primary key.

DynamoDB uses the partition key value as input to an internal hash function. The output from the hash function determines the partition (physical storage internal to DynamoDB) in which the item will be stored. All items with the same partition key value are stored together, **in sorted order** by sort key value.


{
"AnimalType": "Dog",
<…other attributes…>
}
f(x)
Hash Function
{
"AnimalType": "Fish",
<…other attributes…>
}
{
"AnimalType": "Lizard",
<…other attributes…>
}
{
"AnimalType": "Bird",
<…other attributes…>
}
{
"AnimalType": "Cat",
<…other attributes…>
}
{
"AnimalType": "Turtle",
<…other attributes…>
}
{
"AnimalType": "Dog",
<…other attributes…>
}
Partition
Partition
Partition

With Sort Key (Name)



Secondary Indexes

A secondary index provides **fast access** and lets you query the data in the table using an **alternate key**, in addition to queries against the primary key.

GameScores

UserId	GameTitle	TopScore	TopScoreDateTime	Wins	Losses	
"101"	"Galaxy Invaders"	5842	"2015-09-15:17:24:31"	21	72	…
"101"	"Meteor Blasters"	1000	"2015-10-22:23:18:01"	12	3	…
"101"	"Starship X"	24	"2015-08-31:13:14:21"	4	9	…
"102"	"Alien Adventure"	192	"2015-07-12:11:07:56"	32	192	…
"102"	"Galaxy Invaders"	0	"2015-09-18:07:33:42"	0	5	…
"103"	"Attack Ships"	3	"2015-10-19:01:13:24"	1	8	…
"103"	"Galaxy Invaders"	2317	"2015-09-11:06:53:00"	40	3	…
"103"	"Meteor Blasters"	723	"2015-10-19:01:13:24"	22	12	…
"103"	"Starship X"	42	"2015-07-11:06:53:00"	4	19	…
…	…	…	…	…	…	

GameTitleIndex

GameTitle	TopScore	UserId
"Alien Adventure"	192	"102"
"Attack Ships"	3	"103"
"Galaxy Invaders"	0	"102"
"Galaxy Invaders"	2317	"103"
"Galaxy Invaders"	5842	"101"
"Meteor Blasters"	723	"103"
"Meteor Blasters"	1000	"101"
"Starship X"	24	"101"
"Starship X"	42	"103"
...	...	...

If you need an item (game score) by GameTitle, you would need to **scan** every single record that is inefficient as your table gets bigger.

To speed up, you can create an index with GameTitle along with sort key which is TopScore. The table's **primary key attributes** (partition key and sort key) are always projected into an index.

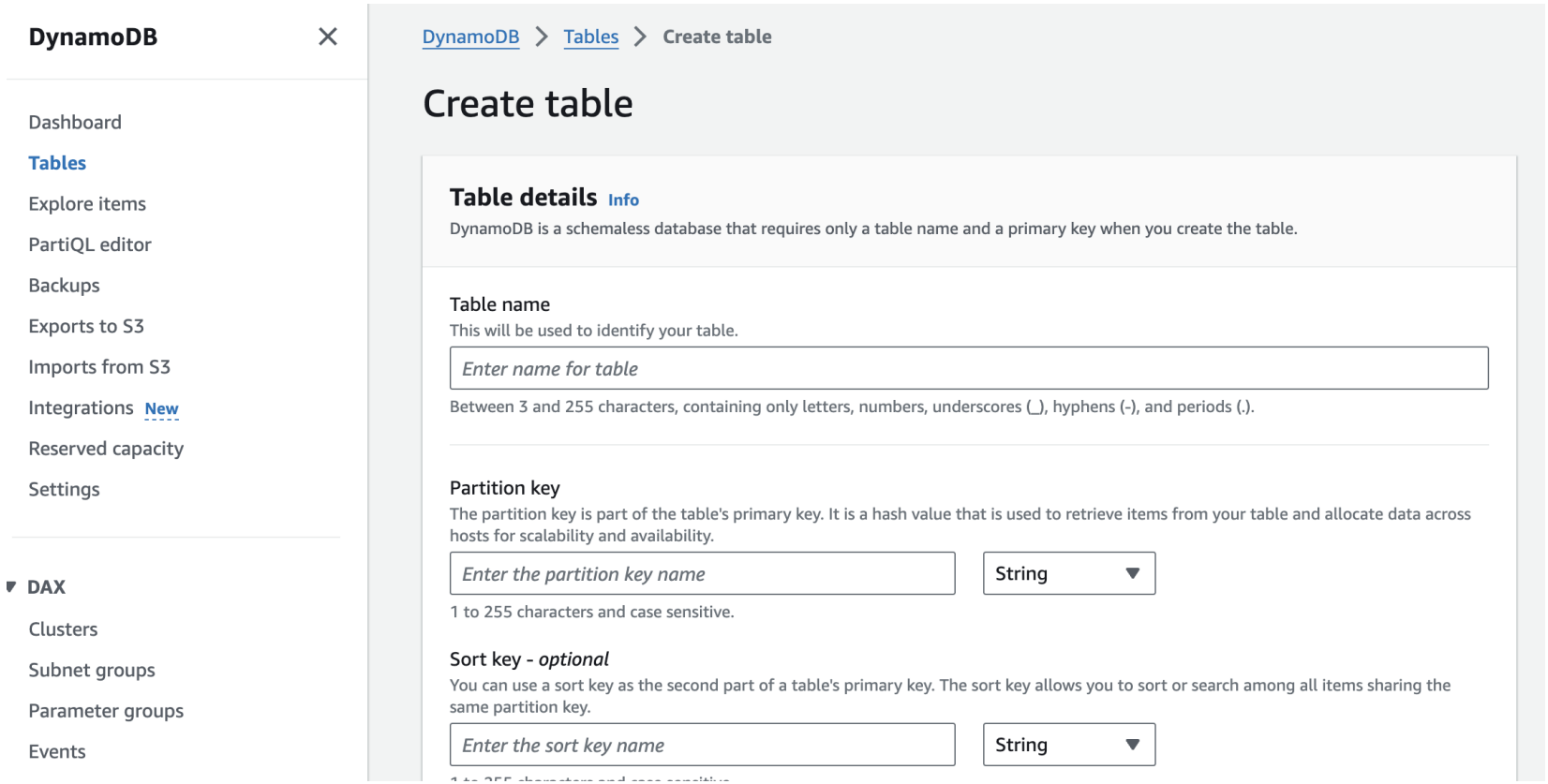
ACID transactions

Amazon DynamoDB transactions simplify the developer experience of making coordinated, **all-or-nothing** changes to multiple items both within and across tables. Transactions provide atomicity, consistency, isolation, and durability (ACID) in DynamoDB, helping you to maintain data correctness in your applications.

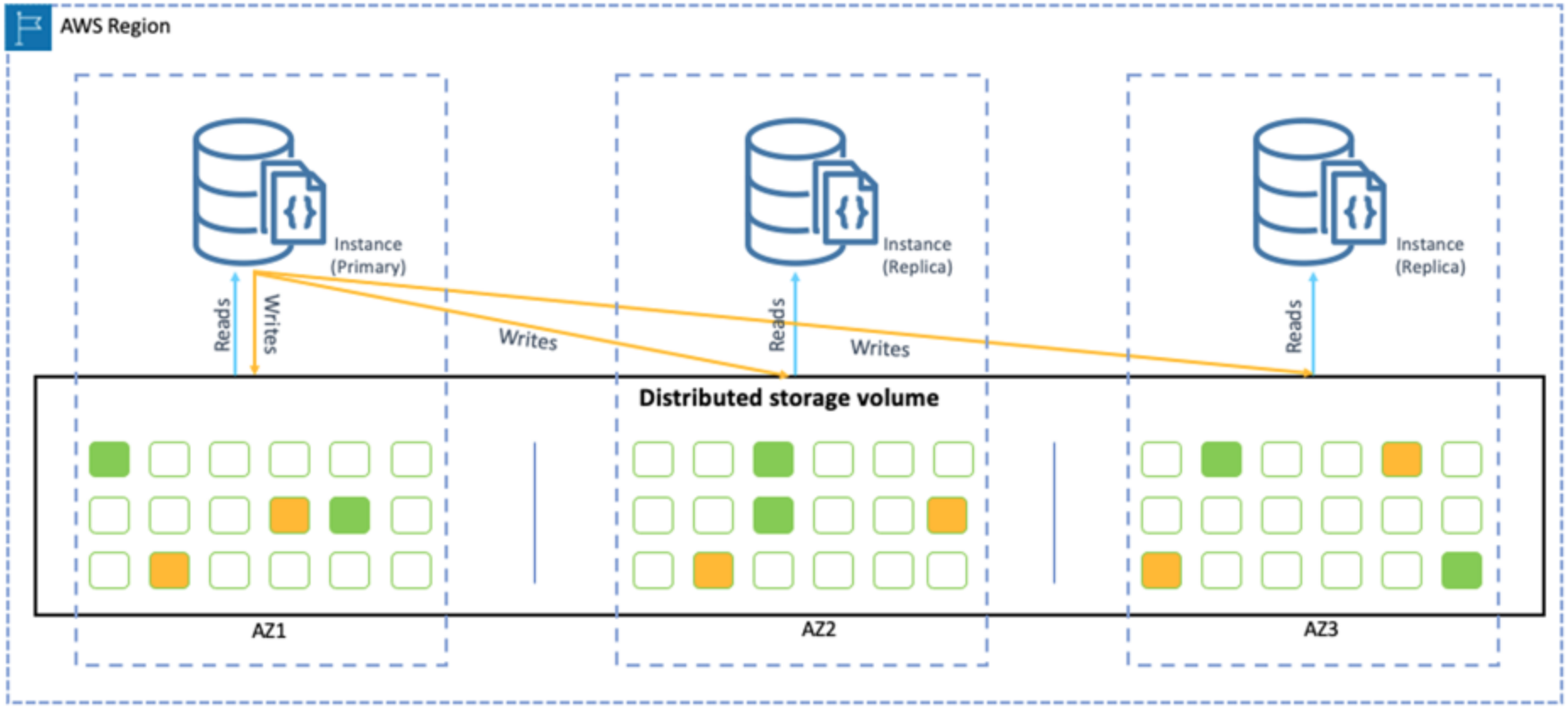
- **TransactWriteItems** - group multiple *Put, Update, Delete*, and *ConditionCheck* actions
- **TransactGetItems** – multiple *Get* actions

Amazon DynamoDB

- Demo



Amazon DocumentDB



Amazon DocumentDB

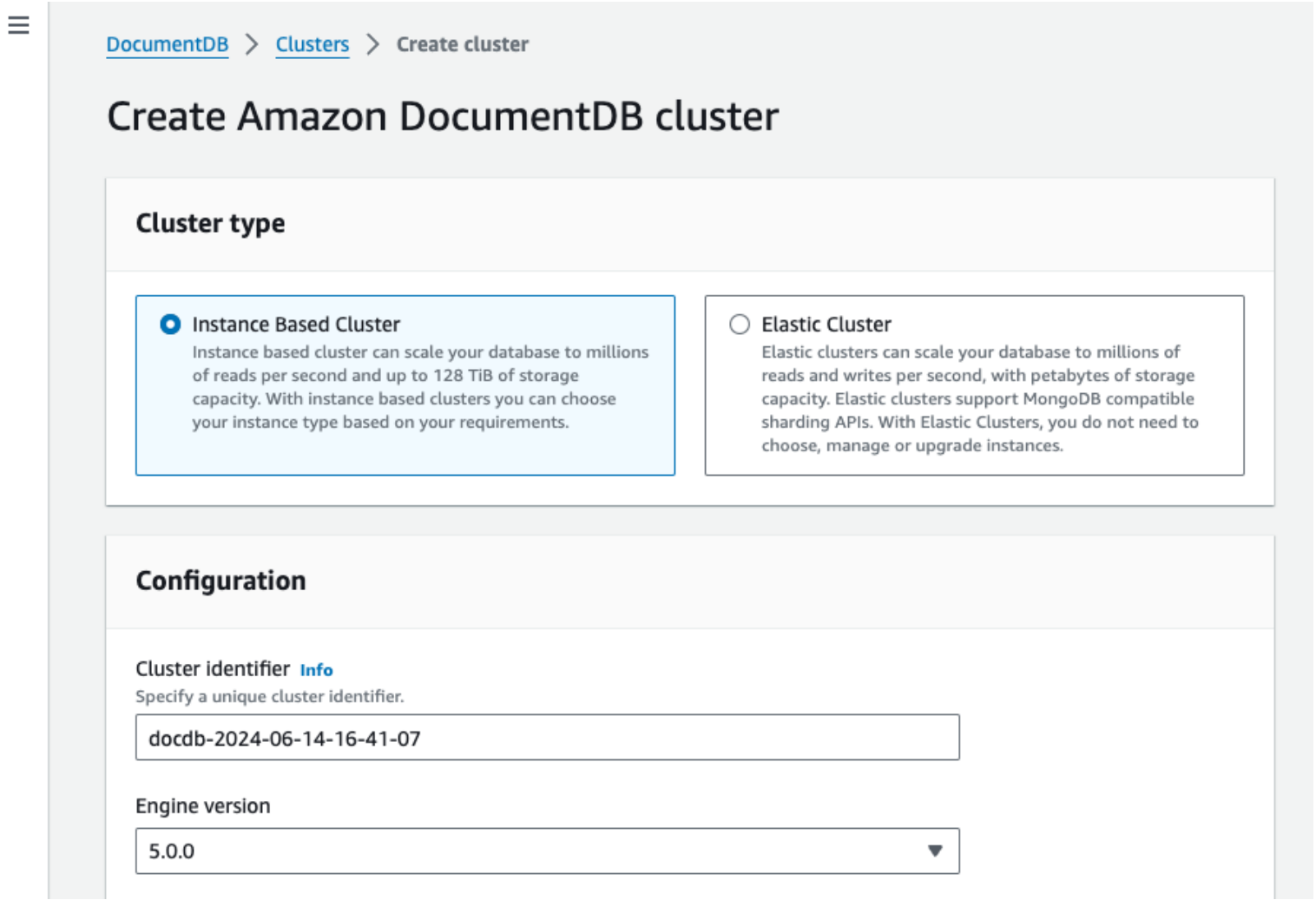
- **Cluster** — A cluster consists of one or more instances that provide the compute, and a cluster volume that manages the data for the instances. A cluster can have up to 16 instances (a primary and up to 15 read replicas). Cluster instances need not be all of the same instance size.
- **Primary instance** — An instance that supports read/write workloads and performs all the data modifications to the cluster volume. Each Amazon DocumentDB cluster has one primary instance.
- **Cluster volume** — The cluster volume provides SSD-backed distributed storage for your database. The primary instance and any Amazon DocumentDB replicas share the same cluster volume.
- **Replicas** — An Amazon DocumentDB replica supports only read operations, and each DB cluster can have up to 15 Amazon DocumentDB replicas. In case the primary instance fails, one of the Amazon DocumentDB replicas is promoted as the primary.

Amazon DocumentDB

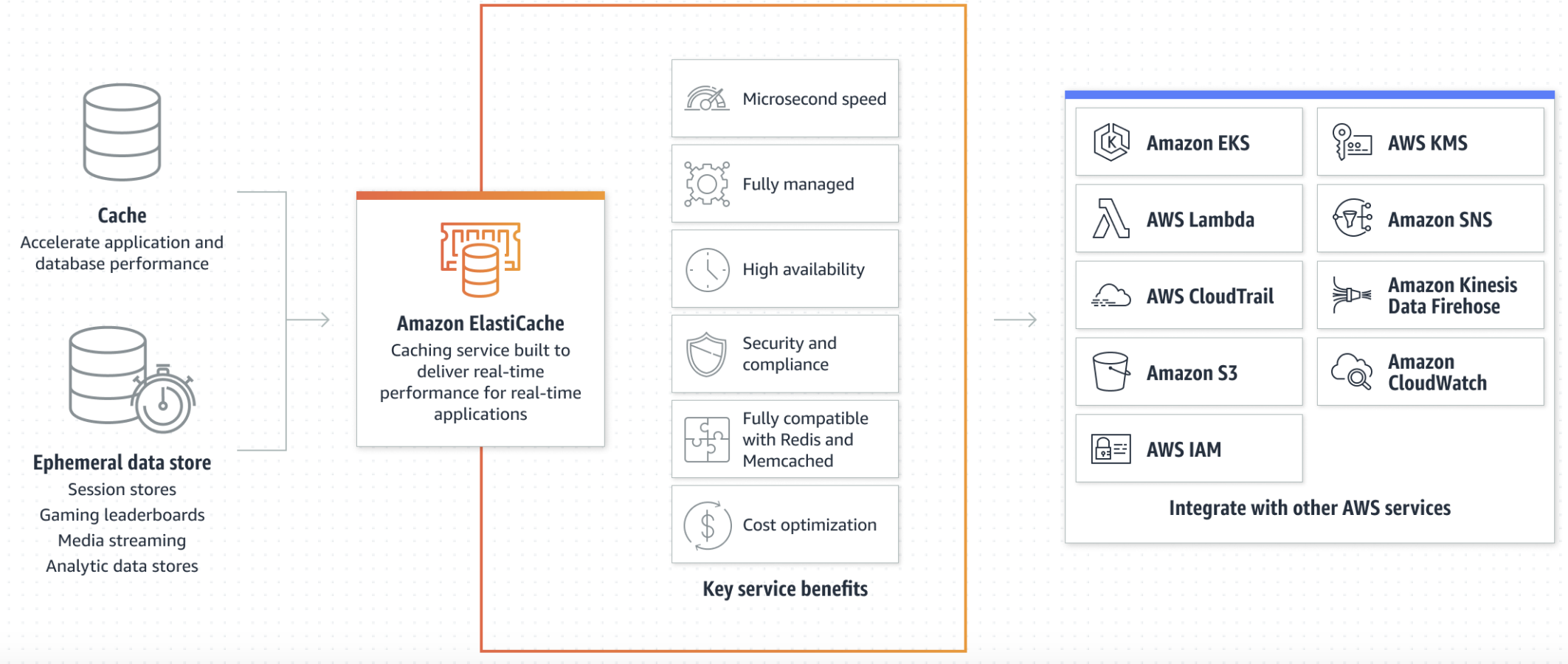
- Fully managed service
- Compatible with MongoDB
- Highly scalable
- Fault tolerant
- High performance
- Highly secure

Amazon DocumentDB

- Demo



Amazon ElastiCache



Amazon ElastiCache

- A managed in-memory cache service in the AWS ecosystem.
- Reducing latency.
- Decreasing database load.
- Improving application response times

Amazon ElastiCache

- Key features
  - Caching strategies
  - Cache Engines: Redis or Memcached
  - Performance

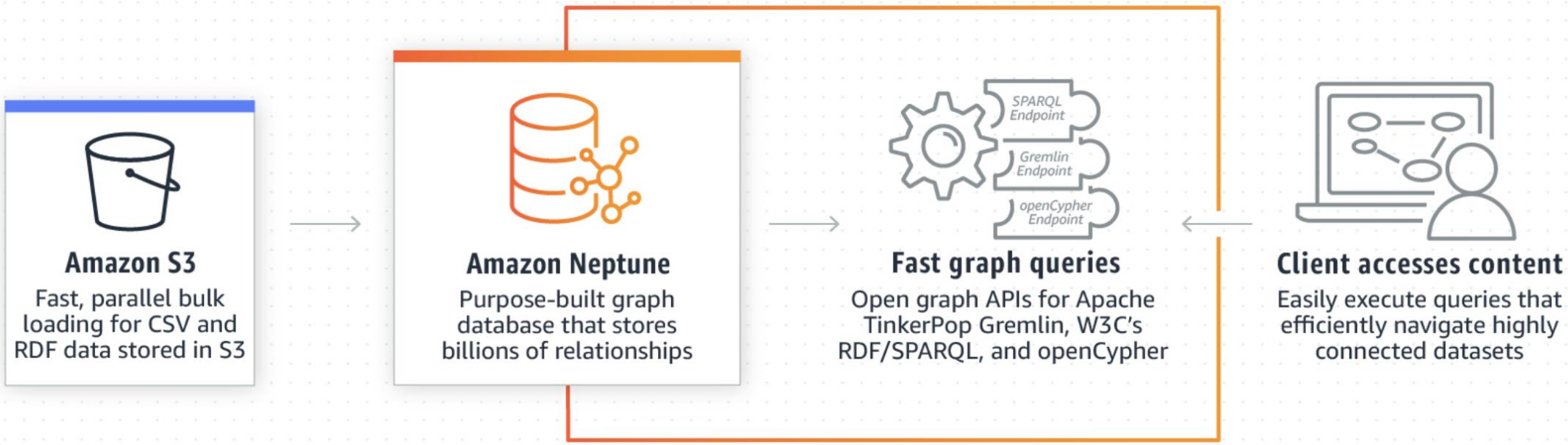
Amazon ElastiCache

- Components
  - Cluster Configuration
  - Security

Amazon ElastiCache

- Use cases
  - Frequently Accessing Data
  - Session Management
  - Database Query Optimization
  - Real-time analytics

Amazon Neptune



Amazon Neptune

- A fully managed graph database service
- Suitable for highly connected data

Amazon Neptune

- Graph Model Supported
- Performance and Scalability
- High Availability and Durability
- Security

Amazon Neptune

- Cluster Configuration: Instances, endpoints, and replicas
- Connectivity: how application connect to Neptune using VPC for isolation

Amazon Neptune

- Use cases:
  - Knowledge graphs: AI applications that require a comprehensive view of customers, business, and relationships.
  - Fraud Detection: Detect complex fraudulent patterns more easily than traditional relational databases.
  - Recommendation Engines: Build recommendation systems that leverage the relationships between data points to deliver more precise recommendations.

References

- https://docs.aws.amazon.com/
- ChatGPT: https://chatgpt.com/
- Google AI: https://gemini.google.com/app
- Redis: https://redis.io/docs/latest/commands